Processing queues, fairness, rate limits and database connections

Design

- **One task per document per step.** An upload enqueues six independent tasks for that document (extraction, metadata forensics, ELA + copy-move + ghost text, duplicate hashing, visual review, signature detection), each carrying the document's company. No task ever processes a whole case or batch.
- **Three shared queues, one per external dependency**, each with its own worker pool:

Queue	Tasks	Limited by	Pool
`extraction_queue`	`process_document` (Document Intelligence layout **+ the Azure OpenAI classification call**, and the spelling check's one call when it has candidates)	`AZURE_DOCUMENT_INTELLIGENCE_MAX_CALLS_PER_SECOND` (10; Azure allows 15) — and, for the classification call, the Azure OpenAI caps	threads
`vision_queue`	visual review, signature detection/comparison, field validation + issuer check	`AZURE_OPENAI_MAX_REQUESTS_PER_MINUTE`, `AZURE_OPENAI_MAX_TOKENS_PER_MINUTE`	threads
`forensics_queue` (+ legacy `celery`)	metadata forensics, ELA/copy-move/ghost text, duplicate hashing, cross-document check, risk scoring, bulk-zip ingestion	**nothing** — local CPU (exception: when the ghost-content check finds a deleted text block — rare — it makes a few Document Intelligence Read and Azure OpenAI calls for the reviewer's hint, through the same global rate limiters)	**prefork**, one process per CPU of the container

  Platform jobs run on a fourth, separate **`housekeeping_queue`**: `log_queue_metrics` (every minute), `requeue_stuck_documents` (every 5 minutes) and `reconcile_usage_stats` (nightly). Its only consumer is the beat container (`start-beat.sh`: beat + one `solo` worker slot). Because the every-minute job no longer lands on `forensics_queue`, the forensics workers see work only when documents arrive, so they can **scale to zero** when idle (KEDA on the forensics lists, `minReplicas: 0`; a Redis list-length scaler activates on the first waiting task). `housekeeping_queue` is not one of the three processing queues in the monitor.
- **Stuck-document recovery** (`app/tasks/stuck_documents_task.py`). Queued tasks live only in Redis. If Redis restarts without its data, or is unreachable at upload time (`enqueue_document_pipeline` then logs and returns instead of failing the upload; the document is already stored), a document would stay `pending` (or `processing`) forever. Every 5 minutes, **only while the extraction queue is empty and idle**, the job re-queues the whole pipeline for documents `pending` > `STUCK_DOCUMENT_PENDING_MINUTES` (10) or `processing` with no update for > `STUCK_DOCUMENT_PROCESSING_MINUTES` (30). A document's first task is `process_document` on the extraction queue, so an empty, idle extraction queue means that task is not coming; a long backlog therefore never causes duplicate work. Each re-queue writes a `document_requeued` audit row; after `STUCK_DOCUMENT_MAX_REQUEUES` (3) the document is marked `failed` with an explanation. Re-running is safe because every step writes idempotently (below). The run logs a WARNING `stuck_documents_requeued {json}` line when it acted. Not covered: a document whose extraction finished but whose other queued checks were lost; those checks stay missing until the document is re-uploaded.
- **Fair-share per company** (`app/tasks/fairshare.py`). Each task is published with a priority computed from how much work *its company* already has outstanding (queued + running) on that queue: `priority = min(9, outstanding // FAIR_SHARE_BUCKET_SIZE)` (bucket default 5). Celery's Redis transport keeps one list per level

(`<queue>`, `<queue>:1` … `<queue>:9`) and workers always drain level 0 first, so a company with little outstanding work is served before the tail of another company's big batch. Equal share, no fixed tiers: priorities are recomputed for every task from live counters, and a company's new tasks move forward again as its backlog drains. Counters are incremented on publish and decremented in `task_postrun`; a redelivered task (worker crash) is counted once; the per-minute metrics job resets a queue's counters whenever that queue is empty (self-healing). If Redis is unreachable a task is published without priority (FIFO) — scheduling degrades, processing doesn't stop.

- **Fair dispatch per worker**: `worker_prefetch_multiplier=1` + `task_acks_late` — a worker holds only the task it is running.
- **Idempotent writes** (`app/services/check_store.py`): exactly one `document_checks` row per (document, check_type), written by `INSERT … ON CONFLICT DO UPDATE`; likewise one page hash per (document, page), one assessment per (case, evidence fingerprint), one signature match per (reference, document, scope) — all enforced by unique constraints. A task redelivered after a worker crash overwrites instead of duplicating; the full run history stays in the append-only audit log.
- **No transaction held during slow work.** Tasks commit (ending their read transaction) before every slow step — blob download, Azure call, CPU forensics — and task sessions keep loaded attributes after a commit (`expire_on_commit=False`) so touching them doesn't silently reopen one. Through PgBouncer in transaction mode an open transaction pins a server connection; the first Linux load test showed 32 threads holding transactions across Azure calls exhausting the pool and stalling the API. The upload endpoint likewise ends its read transaction before reading the body and writing the blob.
- **Independently scalable workers**: `start-worker.sh extraction|vision|forensics`, or the `*-worker` services in `docker-compose.yml` (`--profile workers`, `--scale forensics-worker=N`). The forensics worker runs prefork with one process per CPU of **its container's cgroup limit** (`cpus:` / `FORENSICS_WORKER_CPUS`), not the host's core count — `nproc` ignores container limits and would oversubscribe (2 replicas each starting 4 processes on a 4-core host). Pool sizes: `EXTRACTION_WORKER_CONCURRENCY`, `VISION_WORKER_CONCURRENCY`, `FORENSICS_WORKER_CONCURRENCY` (0 = per CPU of the limit).
- **Monitoring**: Platform › Processing queues (`GET /platform/queues`) — waiting (all priority levels), running, oldest waiting task, average / p95 / max wait, run time, consumers online, limiter counters (throttled calls, seconds waited, Azure 429s) and **outstanding work per company** (the fair-share counters). A beat task logs one structured line per queue every minute, `queue_metrics {"queue": …, "waiting": …, "running": …, "oldest_waiting_seconds": …, "avg_wait_seconds": …, "p95_wait_seconds": …, "started_in_window": …, "window_seconds": 300}`, and a **WARNING** `queue_alert {…same fields…, "alert": "oldest_waiting_exceeded", "threshold_seconds": …}` line when a queue's oldest waiting task is older than `QUEUE_ALERT_OLDEST_WAITING_SECONDS` (default 300; 0 = off). Shipping these logs to a monitoring platform and alerting on `queue_alert` is a deployment step (not built yet: local dev only). For example, on Azure Container Apps with a Log Analytics workspace, a log alert on `ContainerAppConsoleLogs_CL | where Log_s has "queue_alert"` (production-deployment.md).

Azure rate limits

Global limiter (`app/services/rate_limiter.py`): Redis sliding windows shared by every worker, applied per Azure call (Celery's own `rate_limit` is per worker). A 429 that still gets through pauses every worker for the Retry-After period (the SDKs retry first).

This deployment's quota (from the `x-ratelimit-*` headers of `gpt-4.1-mini`): 100 requests/min, **100,000 tokens/min**. `backend/.env` caps at **80 %** (`80` RPM, `80000` TPM). This was 85 % until 2026-10-03. It was lowered because the real-quota load test saw one 429 in 123 calls at full saturation (two independent sliding windows can race). The load-test figures below were measured at 85 %.

Token reservations are calibrated against real usage (`scripts/calibrate_llm_tokens.py`). Azure charges each request its prompt tokens + its full `max_tokens` up front. Every call now logs its real `usage` next to the reservation

(`llm_usage …` log line, Redis `fddt:llm_usage:<call type>`). Measured on the 12 sample documents (English + Arabic, 96 calls):

Call type	Real prompt (avg)	Old estimate	New estimate	Real max output	`max_tokens` old → new
classification + extraction	2,105	1,270 (**0.60×**)	2,762 (1.31×)	1,207	4,000 → 4,000
visual review (2 per page)	2,904	1,564 (**0.54×**)	3,622 (1.25×)	204	1,500 → 640
signature/stamp detection	1,418	1,392	1,967 (1.39×)	144	800 → 480
signature comparison	1,055	2,649 (2.5×)	1,558 (1.48×)	47	800 → 320
issuer entity match	400	353	768 (1.92×)	155	4,000 → 512

- The old estimator (4 chars/token, flat 1,100 tokens/image) **under-reserved the two most frequent calls by 36–46 %**: at full load the limiter would have let traffic exceed the real quota.
- New model: text at **1.93 chars/token** (the most token-dense observed — Arabic; English prompts are reserved a little high, never low) + images at **1.62 tokens per 32-px patch** (how gpt-4.1-family models bill images; full pages and small crops now cost what they really cost) + **5 % margin**. Every measured call is reserved at ≥ 1.05× its real prompt.
- Output reservations were sized from measured outputs with ample headroom for the four fixed-shape calls; classification keeps 4,000 because its output grows with the number of fields on a document.
- **Real-Azure check**: 40 vision calls from 8 threads through the limiter at the 85 % caps → **40 × HTTP 200, 0 × HTTP 429, 0 SDK retries**; the token limiter held 15–20 calls back and settled at 19 calls/min.
- Re-run the calibration when the model, the prompts or the page render size change.

Throughput ceiling at the current quota (what the limiter reserves per one-page document: classification 6.8k + visual review 2 × 4.3k + signature detection ~3 × 2.4k + issuer match 1.3k ≈ 23.9k tokens): **≈ 3.6 documents/minute (~210/hour) for the whole platform.** The earlier "5–6/minute" figure was based on the under-estimates. The `max_tokens` reductions raised the real ceiling by ~26 % (from ≈ 2.8/min). Raising the deployment's TPM is the main lever beyond this.

Database connections

Without PgBouncer (direct)

Each process keeps a SQLAlchemy pool **per role it uses** (app + platform): up to `DB_POOL_SIZE + DB_MAX_OVERFLOW` = 5 + 10 = **15 per role**, opened on demand.

Process	Concurrency that can hold a connection	Max server connections
API (uvicorn worker)	FastAPI sync endpoints run in a 40-thread pool → capped by the pool	2 roles × 15 = **30**
extraction / vision worker (threads, T threads)	T	≈ min(T, 15) app + a few platform
forensics worker process (prefork)	1 task at a time	≈ 1–2

Example: 2 API replicas × 4 uvicorn workers (8 × 30 = 240) + extraction 16 + vision 16 + 2 forensics replicas × 2 processes (≈ 8) ≈ **280** — beyond Postgres' default `max_connections = 100` . Hence:

With PgBouncer (transaction pooling) — the recommended setup

Set `DATABASE_POOLER_HOST` / `DATABASE_POOLER_PORT`: the two app roles connect to PgBouncer (`pgbouncer/pgbouncer.ini`, service `pgbouncer` in `docker-compose.yml`) and **each process keeps no pool of its own** (`NullPool`), so idle slack isn't held twice. Client connections are cheap; PgBouncer multiplexes them onto a small set of server connections:

PgBouncer setting	Value	Meaning
`pool_mode`	`transaction`	a server connection is lent per transaction
`default_pool_size`	20	server connections per (database, user): `fddt_app`, `fddt_platform`
`reserve_pool_size`	5 (after 3 s)	burst headroom per pool
`max_db_connections`	60	hard cap of server connections to the database
`max_client_conn`	1000	client connections accepted (API + workers)

Postgres `max_connections` required: PgBouncer's 60 + direct owner connections for migrations/seed (≈ 2) + admin/monitoring (≈ 5) + superuser reserve (3) ⇒ **≥ 70; the default 100 is enough.** Raise `default_pool_size` / `max_db_connections` (and `max_connections` with them) only if PgBouncer's `SHOW POOLS` shows clients waiting (`cl_waiting`).

Tenant context is safe with transaction pooling because the company is set with `set_config('app.current_company_id', …, is_local => true)` at the start of every transaction and dies with it. `tests/test_pgbouncer_rls.py` proves it on a single-server-connection pool (forced reuse): a *session*-level setting **does** leak to the next client (negative control — the test can detect leaks), the transaction-local one never does, including 32 concurrent clients × 25 transactions alternating companies. Never set tenant state with plain `SET` / `set_config(..., false)`.

Production: `userlist.txt` holds local dev passwords only — use SCRAM secrets from a vault.

Load tests

`backend/loadtest/` drives the real API, real Celery workers, Postgres (RLS) and Redis; only Document Intelligence, Azure OpenAI and Blob Storage are stubbed (they simulate latency and Azure's own quotas, answering 429 above them). `docker-compose.loadtest.yml` runs it all in **Linux containers through PgBouncer**.

Linux, final run (forensics 2 replicas × 2 prefork processes; Azure queues 16 threads; Azure OpenAI quota simulated at 10× so the run takes minutes)

Check	Result
1 — 27 concurrent single uploads, 3 companies	interleaved (`ABCABCABCACBB…`), Kendall τ vs submission 1.0 / 0.86 / 0.95 (extraction / vision / forensics)
2 — A uploads 100 docs, B uploads 1 right after	**B waited 0.0–0.2 s on forensics (A's median 30 s) and 1.2 s on visual review (A's median 44 s)**; started after 43–45 of A's 100 instead of all 100
3 — no 429 storms	Document Intelligence peaked at exactly 10 calls/s (quota 15), **0 × 429**; Azure OpenAI **0 × 429**, token limiter held 242 calls back
4 — monitor reflects reality	peaks 20 / 146 / 299 waiting, tracked every 2 s for the whole run
Drain	everything drained in 92 s; forensics finished before vision (97 s vs 130 s)

Linux, real quota (85 RPM / 85k TPM app caps vs 100 / 100k simulated Azure), 30-document batch

Queue	Last task started	Oldest wait
forensics	**35 s**	16 s
extraction	69 s	50 s
vision	315 s	288 s

At the real quota **forensics is the fastest-draining queue by far** and the Azure OpenAI quota is the long pole — as it should be. Fair-share: B started after 10 of A's 30 on forensics and vision. One simulated 429 occurred in 123 calls while the token limiter was fully saturated (103 calls held back) — a boundary race between the app's window and the stub's independent window; the real SDK retries such a 429 after Retry-After (the stub doesn't), and the real-Azure burst at the same caps gave none. For more margin set the caps to 80 % of quota.

Comparison with the first (Windows, strict FIFO) run

	Windows threads, FIFO	Linux prefork, fair-share
B's wait behind A's 100-doc batch (forensics / vision)	144 s / 70 s, after **100/100**	0.2 s / 1.2 s, after 43–45/100
Forensics oldest waiting	264 s	47 s (10× load), 16 s (real quota)
429s	0	0

Open findings

- `process_document` **couples Document Intelligence and the Azure OpenAI classification in one task.** When the token limiter is saturated, extraction threads sit blocked on the *OpenAI* quota, so OCR for a newly arrived document waits for a free thread (B waited 14 s / 50 s on extraction in the two runs while A's OCR barely waited). Fix: move classification + extraction into its own `vision_queue` task (needs the OCR word boxes persisted between the two). Not done in this change.
- **A rate-limited task holds its worker thread while it waits for the limiter.** Fair-share decides the dequeue order, but a newly prioritised task still needs a free thread. Sizing the Azure pools to just saturate the caps (not far above) keeps that wait short.
- **On one machine, forensics competes with the API for CPU.** Prefork saturates its CPU allowance by design; run workers and the API on separate nodes (or cap `FORENSICS_WORKER_CPUS`) so uploads stay fast.